

MAPMATE Software Architecture (Enhanced Hybrid Edition)

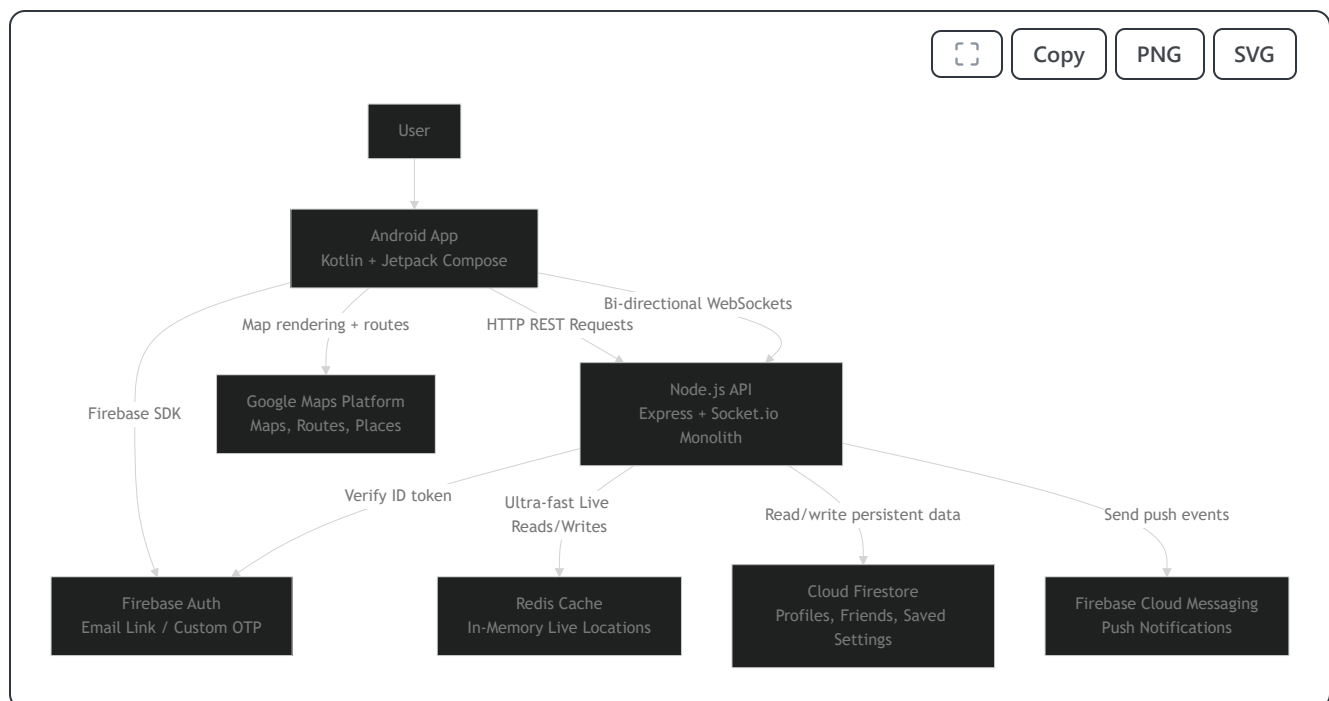
Recommended Architecture Type

MAPMATE uses a **hybrid client-server architecture**:

- **Android client:** MVVM + Clean Architecture style boundaries.
- **Backend:** Node.js modular monolith combining a **REST API** for standard operations and **WebSockets (Socket.io)** for real-time tracking.
- **Data Layer Cache:** An in-memory **Redis** cache layer sits in front of **Cloud Firestore** to handle high-frequency location data without inflating cloud costs.
- **Managed services:** Firebase Auth, Cloud Firestore, Firebase Cloud Messaging (FCM), Crashlytics, and Google Maps APIs.

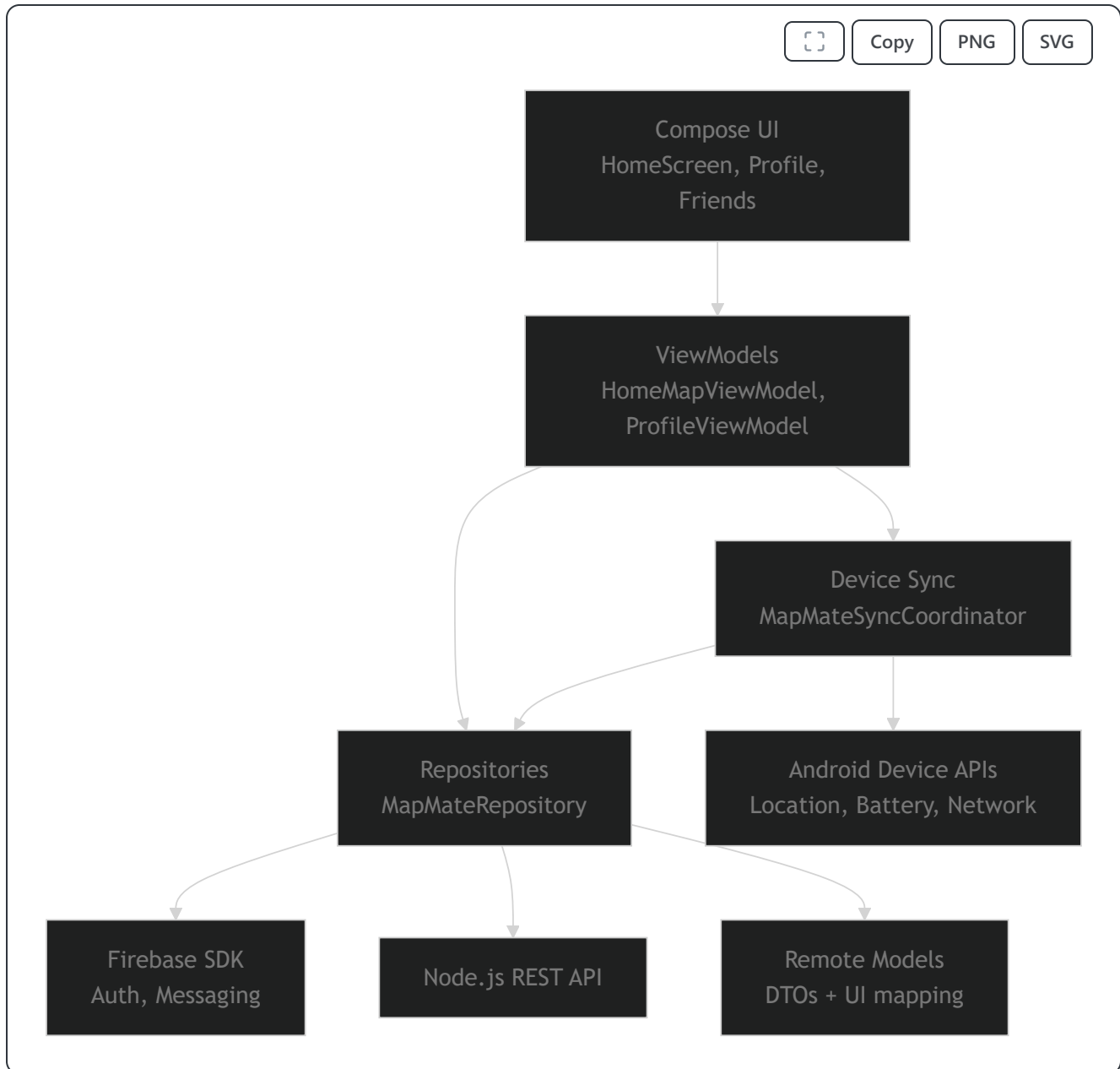
This hybrid approach ensures optimal mobile battery performance and cost efficiency while keeping the deployment boundaries simple within a modular monolith.

High-Level System Diagram



Mobile App Architecture

The Android app should follow **MVVM with repository boundaries**.

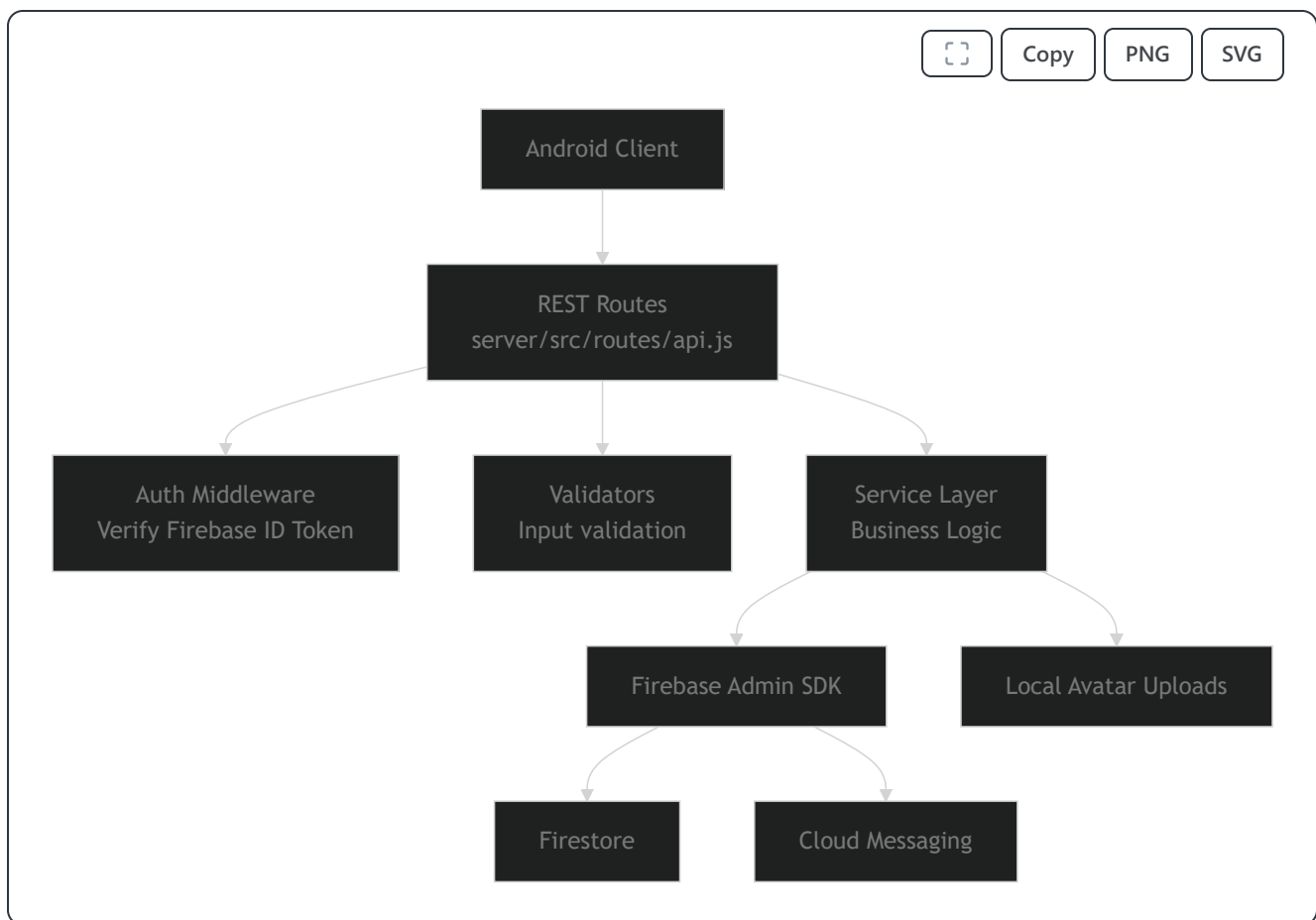


Mobile Layer Responsibilities

Layer	Responsibility
UI	Render map, friend markers, privacy controls, profile and invite screens.
ViewModel	Hold screen state, loading state, errors, and user actions.
Repository	Talk to Node API and Firebase Auth securely.
Sync Coordinator	Send location every 3 seconds when active and battery every 2 minutes.
Device APIs	Location, battery, connectivity, notifications, background service.

Backend Architecture

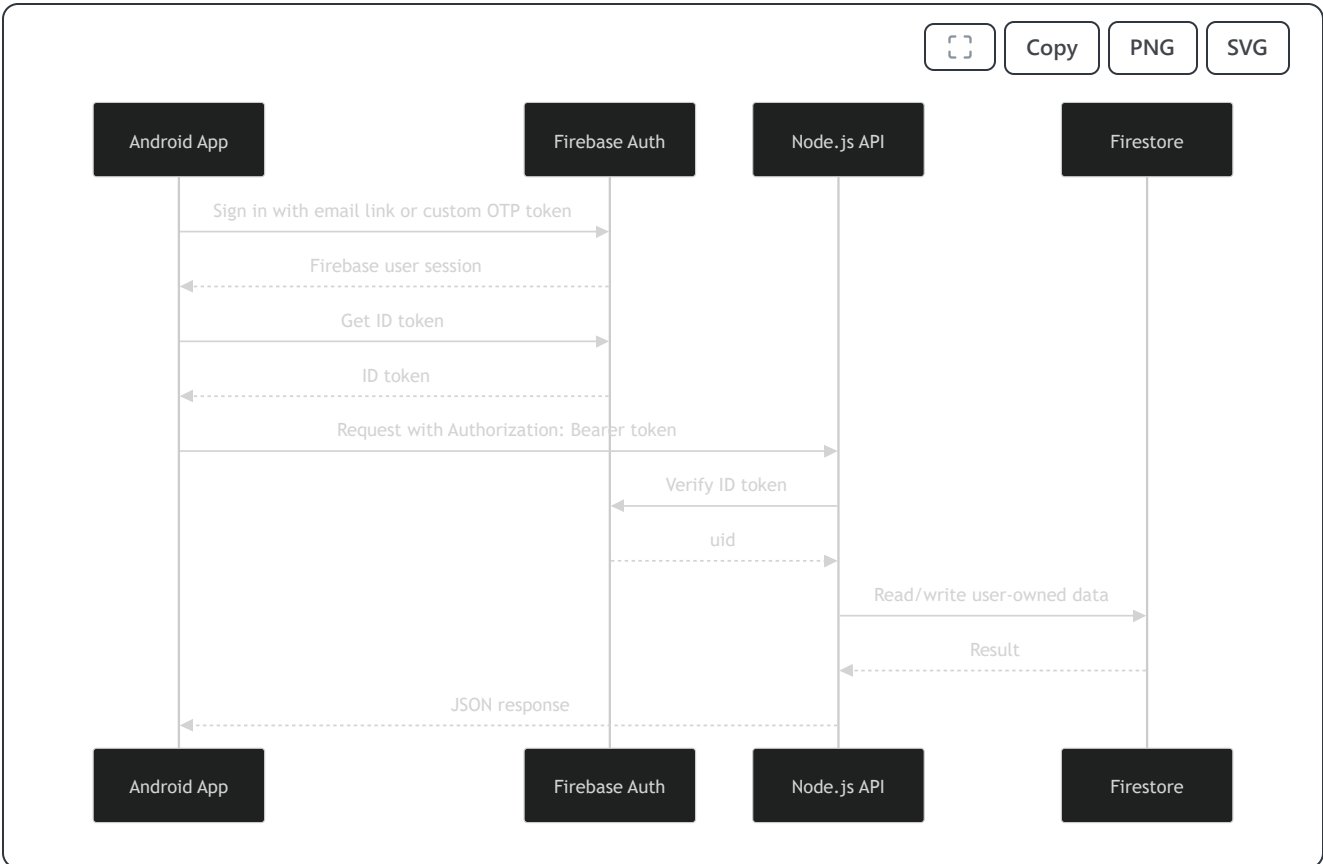
The Node.js backend should stay as a **modular monolith**.



Backend Module Responsibilities

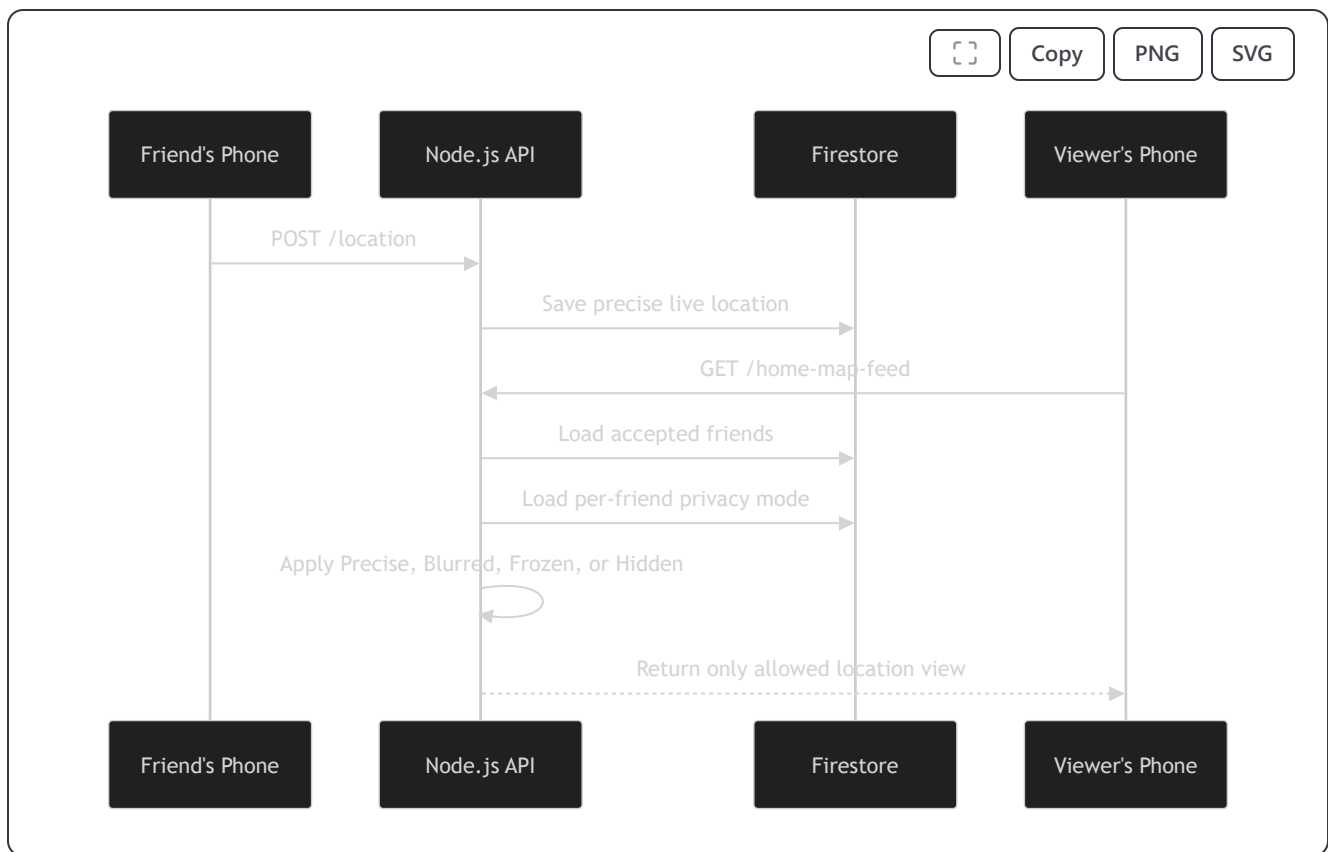
Module	Responsibility
<code>routes/api.js</code>	HTTP endpoints and request/response shape.
<code>middleware/auth.js</code>	Verify Firebase ID tokens from Android.
<code>services/mapmateService.js</code>	Friends, privacy, location, battery, places, notifications.
<code>services/otpService.js</code>	Numeric email OTP request and verification.
<code>utils/validators.js</code>	Input validation and request safety.
<code>firebase.js</code>	Firebase Admin initialization.
<code>errors.js</code>	Consistent API error responses.

Core Request Flow

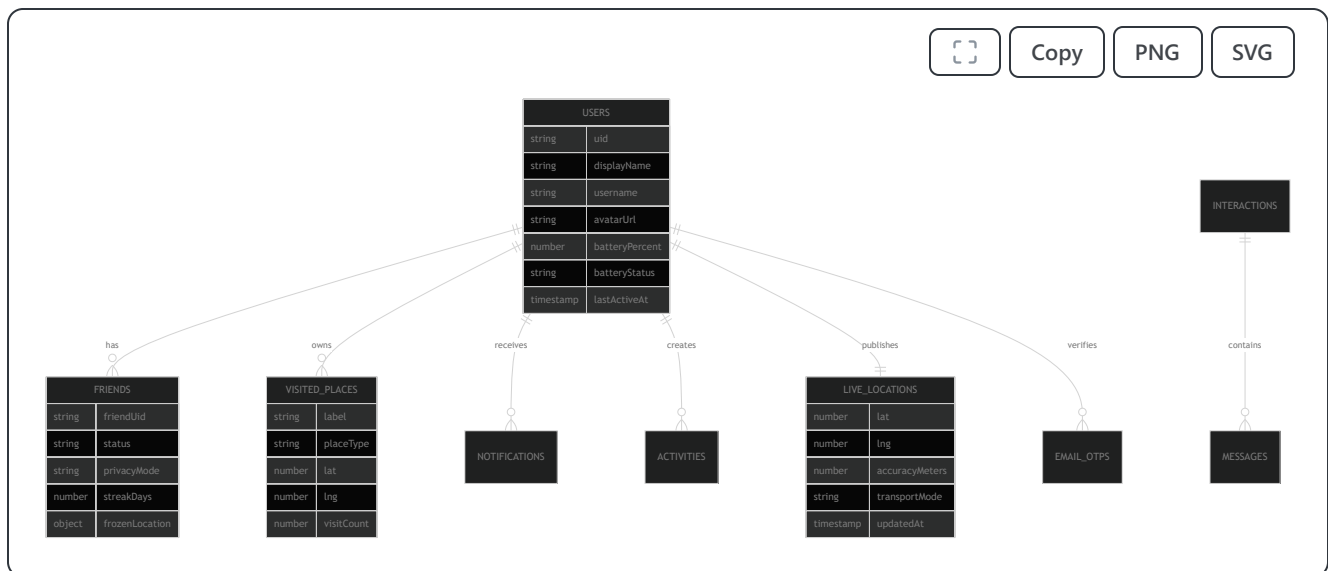


Location And Privacy Flow

Location privacy must be enforced on the backend, not only in the app.



Main Data Collections



API Surface

Feature	Endpoint
Health check	GET /api/health
Numeric OTP request	POST /api/auth/email-otp/request
Numeric OTP verify	POST /api/auth/email-otp/verify
Create profile	POST /api/profile
Update profile	PATCH /api/profile
Upload avatar	POST /api/profile/avatar
Search users	GET /api/users/search
Friend request	POST /api/friends/request
Accept friend	POST /api/friends/accept
Unfriend	POST /api/friends/unfriend
Block user	POST /api/friends/block
Update Ghost Mode	PATCH /api/friends/privacy
Update location	POST /api/location
Update battery	POST /api/battery
Home map feed	GET /api/home-map-feed
Send emoji	POST /api/emoji
Record visited place	POST /api/visited-places
Get visited places	GET /api/visited-places
Notifications	GET /api/notifications
Recent activity	GET /api/activities/recent

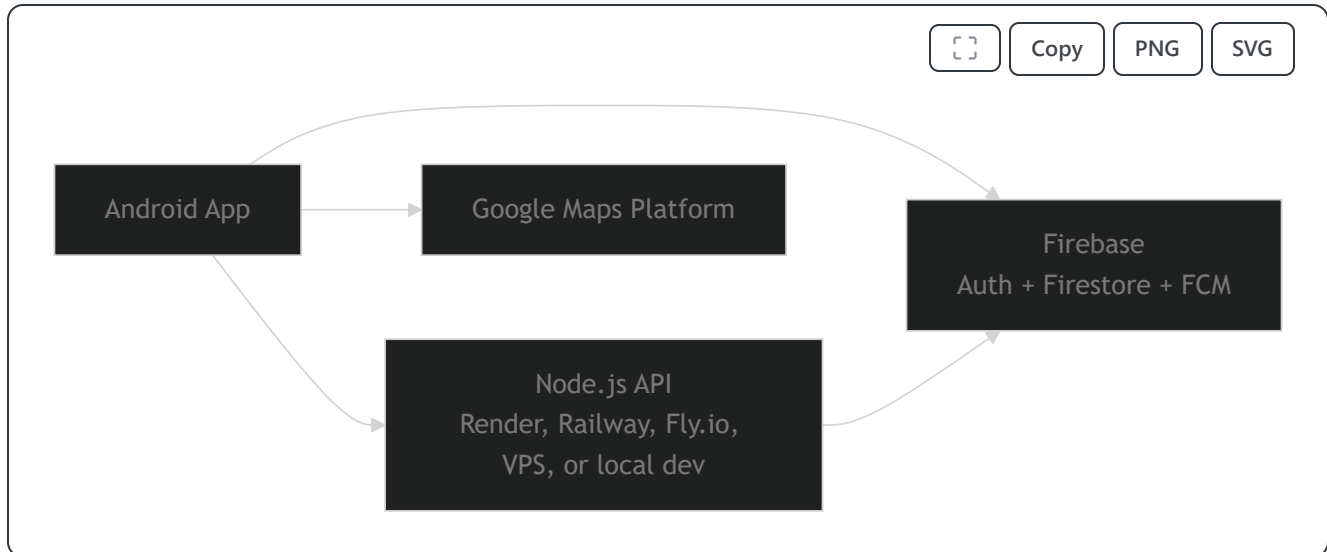
Security Principles

- Android never reads another user's raw live location directly.
- Node.js verifies Firebase ID tokens on every protected endpoint.
- Ghost Mode transformation happens in Node.js before data returns to the viewer.
- Firestore rules should deny direct reads to sensitive collections like `liveLocations`.

- Avatar files and service account files must not be committed.
- Numeric OTP codes are hashed before storage.
- API keys and service credentials stay in local environment files.

Deployment Shape

For MVP, the cheapest practical deployment is:

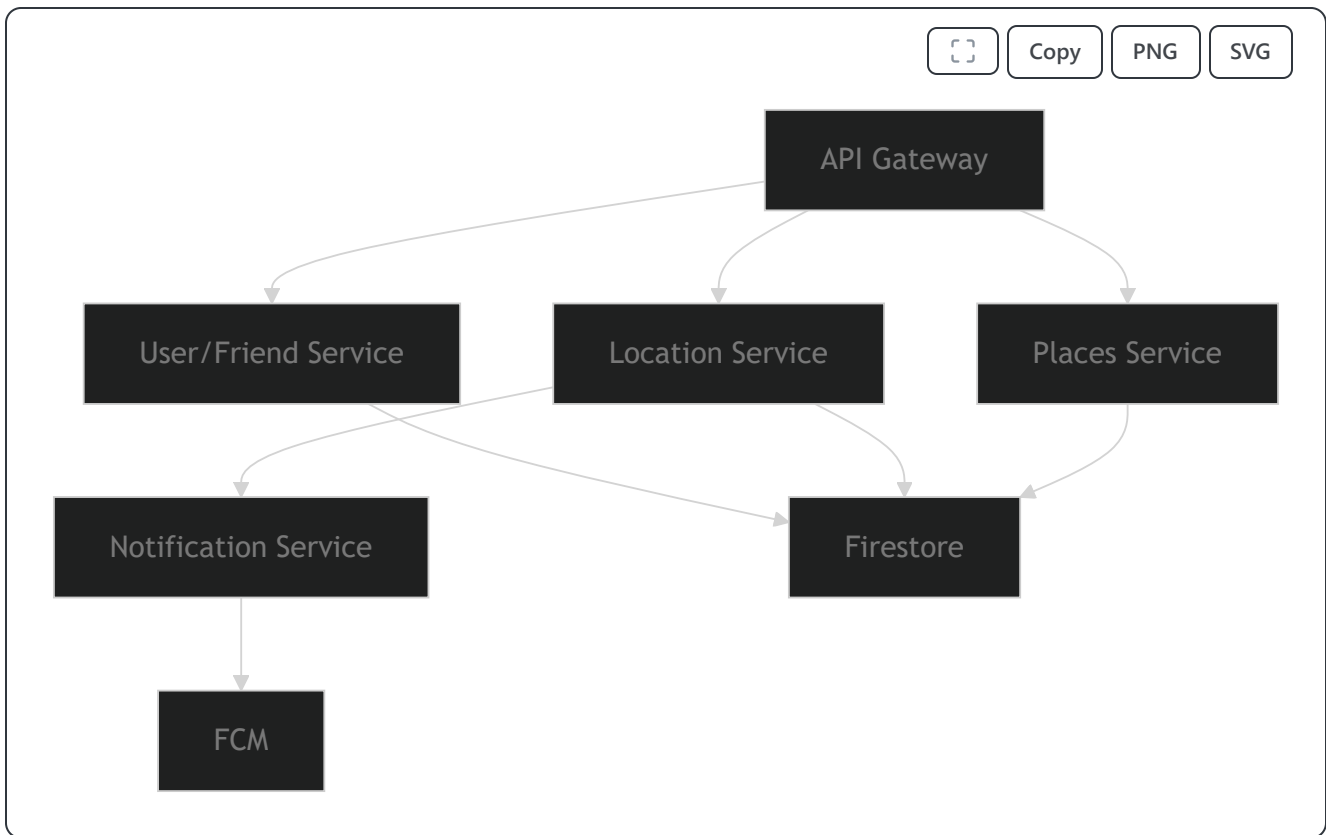


deployment:

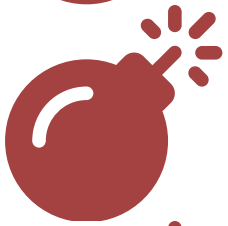
- Local development: Node API on `localhost:8080` .
- Android emulator: call `http://10.0.2.2:8080/api` .
- Real device testing: call your computer LAN IP.
- Production MVP: deploy Node API to a low-cost host.

Future Evolution

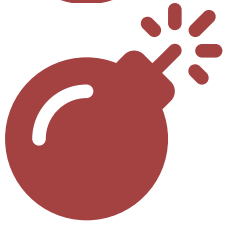
When MAPMATE grows, here is the suggestion to split only the parts that truly need independent scaling:



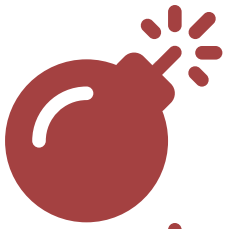
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



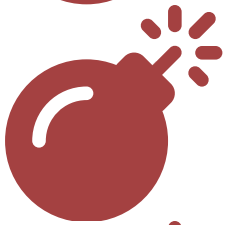
Syntax error in text
mermaid version 11.13.0



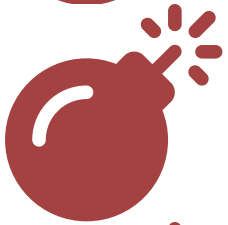
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



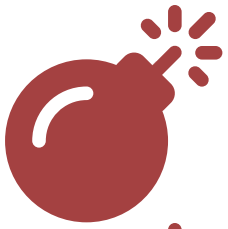
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



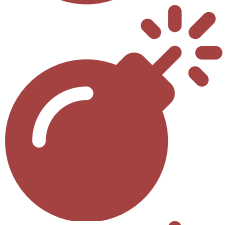
Syntax error in text
mermaid version 11.13.0



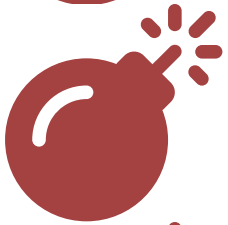
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



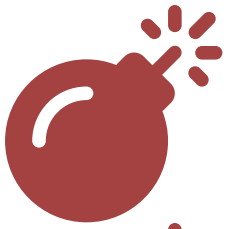
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



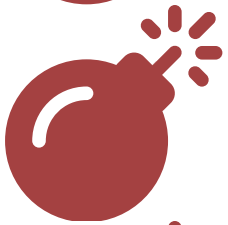
Syntax error in text
mermaid version 11.13.0



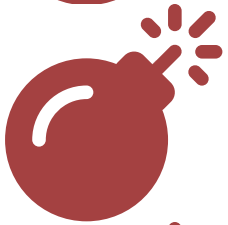
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



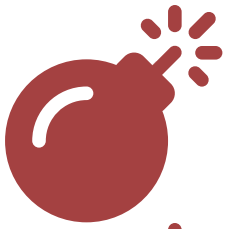
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



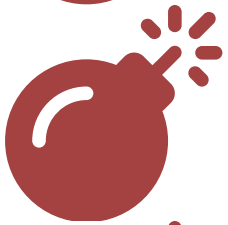
Syntax error in text
mermaid version 11.13.0



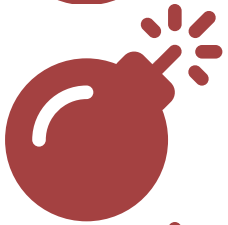
Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0



Syntax error in text
mermaid version 11.13.0